

漏洞分析报告：v4alpha（有漏洞版） vs v4beta（安全修复版）

一、本次重点修复 — 越权访问与权限控制缺失

漏洞1：水平越权 — 通过URL参数查看任意用户资料

漏洞原理

v4alpha 版本的 `/profile` 路由从 URL 查询参数 `user_id` 获取用户标识，并在数据库中查询该标识对应的用户信息，**完全没有验证当前登录用户与所请求用户身份是否匹配**。这意味着任意登录用户只需修改 URL 中的 `user_id` 参数值，即可查看系统中任意用户的敏感信息（邮箱、手机号、余额）。

相比之下，v4beta 从 `session["uid"]` 获取当前登录用户的身份标识，始终只能查看自己的个人信息，不存在越权可能。

漏洞位置

v4alpha/app.py 第 919-938 行：

```
@app.route("/profile", methods=["GET", "POST"])
def profile():
    # DELIBERATE VULNERABILITY: Read user_id from URL parameter, not session
    session_uid = session.get("uid")
    if not session_uid:
        return redirect(url_for("login"))

    user_id = request.args.get("user_id")
    if user_id:
        try:
            user_id = int(user_id)
        except (ValueError, TypeError):
            user_id = session_uid
    else:
        user_id = session_uid

    # DELIBERATE VULNERABILITY: Query the requested user_id, not the session
    user
    # No identity match check
    user = db_get_user_safe(user_id)
```

第 927-934 行是关键：`request.args.get("user_id")` 从 URL 参数中读取目标用户 ID，而第 938 行使用该 ID 查询用户信息，**未做任何身份匹配检查**。

攻击方式

- 普通用户 `alice` 登录系统 (密码 `Alice@2025#UserNew`)
- `alice` 在浏览器地址栏输入 `/profile?user_id=1`
- 页面显示管理员 `admin` 的详细信息:

字段	值
UID	1
用户名	admin
角色	admin
邮箱	admin@example.com
手机	13800138000
余额	99999

- `alice` 成功查看了 `admin` 的敏感信息, 属于 **水平越权 (IDOR, Insecure Direct Object Reference)**

修复方案

`v4beta/app.py` 第 919-928 行 (修复后) :

```
@app.route("/profile", methods=["GET", "POST"])
def profile():
    uid = session.get("uid")
    if not uid:
        session.clear()
        return redirect(url_for("login"))

    user = db_get_user_safe(uid)
    if not user:
        session.clear()
        return redirect(url_for("login"))
```

修复代码对比

关键点	v4alpha (有漏洞)	v4beta (已修复)
用户标识来源	<code>request.args.get("user_id")</code> (URL参数)	<code>session.get("uid")</code> (会话)
身份校验	无, 直接查询任意 <code>user_id</code>	自动绑定到登录用户
未登录访问	检查 <code>session</code> , 但仍可查任意用户	立即重定向到登录页
越权风险	高, 任意登录用户可查看所有用户信息	无, 只能查看自己的信息

效果验证

```
=== 攻击前 ===
alice 登录后访问 /profile → 显示 alice 的信息（余额：100）
alice 访问 /profile?user_id=1 → 显示 admin 的信息（余额：99999，邮箱：
admin@example.com）

=== 修复后 ===
alice 登录后访问 /profile → 显示 alice 的信息（余额：100）
alice 访问 /profile?user_id=1 → 忽略 user_id 参数，仍显示 alice 自己的信息
```

漏洞2：越权充值 — 通过表单参数为任意用户充值/扣款

漏洞原理

v4alpha 的 `/recharge` 路由从 **HTTP POST 表单** 中读取 `user_id` 字段作为充值目标用户，而不是从 `session["uid"]` 获取。虽然页面模板中该字段以 `<input type="hidden">` 形式嵌入，但攻击者可以：

1. 通过浏览器开发者工具修改隐藏字段的值
2. 直接构造 HTTP POST 请求提交任意 `user_id`
3. 使用自动化工具（如 curl、Burp Suite）发送自定义请求

漏洞位置

v4alpha/app.py 第 1033-1046 行：

```
@app.route("/recharge", methods=["POST"])
def recharge():
    # 登录检查（仅确认已登录）
    uid = session.get("uid")
    if not uid:
        return redirect(url_for("login"))

    # DELIBERATE VULNERABILITY: Read target user_id from form, not session
    target_uid_str = request.form.get("user_id", str(uid))
    try:
        target_uid = int(target_uid_str)
    except (ValueError, TypeError):
        return redirect(url_for("login"))
```

第 1042 行 `request.form.get("user_id", str(uid))` 从表单读取目标用户 ID，第 1079-1080 行使用此 ID 执行余额更新：

```
# DELIBERATE VULNERABILITY: Use target_uid from form, not session uid
success = db_update_balance(target_uid, amount)
```

攻击方式

场景：alice 为 admin 扣款（盗取资金）

1. alice 登录系统
2. alice 访问 `/profile?user_id=1` 获取 CSRF Token
3. alice 发送 POST 请求到 `/recharge`：

```
POST /recharge HTTP/1.1
csrf_token=[token]
user_id=1
amount=-99999
```

4. 系统执行 SQL：`UPDATE users SET balance = balance + (-99999) WHERE uid = 1`
5. admin 的余额从 99999 变为 0（减少了 99999）
6. 进一步攻击可继续使用 `amount=-100` 使 admin 余额变为负数（-100）

v4alpha/templates/profile.html 第 99-102 行（隐藏字段来源）：

```
<form method="POST" action="{{ url_for('recharge') }}">
  <input type="hidden" name="csrf_token" value="{{ csrf_token }}">
  <!-- DELIBERATE VULNERABILITY: Hidden user_id field allows targeting any user -->
  <input type="hidden" name="user_id" value="{{ user.uid }}">
```

该隐藏字段的值由模板引擎根据 `user.uid` 渲染。当访问 `/profile?user_id=1` 时，`user` 对象为 admin，因此 `value="1"`。攻击者可直接修改该值。

修复方案

v4beta/app.py 第 1018-1270 行（修复后）：

```
@app.route("/recharge", methods=["POST"])
def recharge():
    """安全余额充值，仅接受 POST 请求。身份仅从 session['uid'] 获取。"""
    uid = session.get("uid")
    if not uid:
        return redirect(url_for("login"))

    # ... 验证和充值操作使用 uid（来自session）...
    success = db_update_balance(uid, amount)
    return redirect(url_for("profile"))
```

修复代码对比

关键点	v4alpha（有漏洞）	v4beta（已修复）
目标用户来源	<code>request.form.get("user_id")</code> （表单参数）	<code>session.get("uid")</code> （会话）
攻击者可指定目标	是，可修改表单或直接 POST 任意 user_id	否，始终操作自己的账户

攻击方式

场景1：负金额扣款

- 1. alice 登录后访问 `/profile?user_id=1`
- 2. 提交 POST 请求到 `/recharge` :

```
user_id=1&amount=-100
```

- 3. 执行 SQL: `UPDATE users SET balance = balance + (-100) WHERE uid = 1`
- 4. admin 的余额减少 100

场景2：零值充值

- 1. 提交 `amount=0`
- 2. 虽然不改变余额，但绕过了业务逻辑应有的校验
- 3. 可能被用于刷充值记录或触发业务系统中的其他副作用

场景3：大额充值

- 1. 提交 `amount=999999999`
- 2. 直接给目标账户增加接近十亿余额
- 3. 可能导致整数溢出或系统中的经济体系崩溃

修复方案

v4beta/app.py 第 1051-1058 行:

```
# 严格校验：必须 > 0
if amount <= 0:
    return render_template("profile.html", user=user, greeting=greeting,
                           recharge_error="充值金额必须大于 0")

# 严格校验：必须 <= 100000
if amount > 100000:
    return render_template("profile.html", user=user, greeting=greeting,
                           recharge_error="单次充值金额不能超过 100000")
```

修复代码对比

校验点	v4alpha (有漏洞)	v4beta (已修复)
amount <= 0 检查	无 — 允许负数和零	<code>if amount <= 0: raise error</code>
amount > 100000 检查	无 — 允许任意大额	<code>if amount > 100000: raise error</code>
HTML min 属性	无 min 限制	<code>min="0.01"</code>
HTML max 属性	无 max 限制	<code>max="100000"</code>
金额合法性	任意数值均接受	仅接受 <code>0 < amount <= 100000</code>

效果验证

```
=== 攻击前 ===
amount=-99999 → 接受, 余额减少 99999
amount=-100   → 接受, 余额减少 100
amount=0      → 接受, 余额不变但不报错
amount=999999 → 接受, 余额大幅增加

=== 修复后 ===
amount=-99999 → 拒绝, "充值金额必须大于 0"
amount=-100   → 拒绝, "充值金额必须大于 0"
amount=0      → 拒绝, "充值金额必须大于 0"
amount=999999 → 拒绝, "单次充值金额不能超过 100000"
amount=100    → 接受, 余额增加 100
```

二、其他同步对比分析

除了上述三个核心漏洞外，以下为 v4alpha 与 v4beta 在安全方面的其他对比项：

2.1 CSRF 保护

比较项	v4alpha	v4beta
CSRF Token 生成	有 (<code>generate_csrf_token()</code>)	有
CSRF Token 校验 (/recharge)	有 (<code>validate_csrf()</code>)	有
CSRF Token 校验 (/profile POST)	有	有
评估	均受保护	均受保护

v4alpha 保留了 CSRF 保护机制，因此攻击者需要获取有效的 CSRF Token 才能实施越权充值攻击。但这**仅增加攻击门槛，并未阻止攻击**——登录用户页面中嵌入了有效 Token，攻击者仍可结合漏洞1先获取目标用户的页面（获取其 CSRF Token），再构造越权充值请求。

2.2 参数化 SQL

比较项	v4alpha	v4beta
SQL 查询方式	参数化 (<code>?</code> 占位符)	参数化 (<code>?</code> 占位符)
SQL 注入风险	无 (使用参数化查询)	无
评估	均安全	均安全

两个版本都使用参数化 SQL，因此不存在 SQL 注入风险。越权攻击并非通过 SQL 注入实现，而是通过**业务逻辑层面的身份校验缺失**实现。

2.3 密码安全

比较项	v4alpha	v4beta
密码哈希	<code>werkzeug.security.generate_password_hash</code>	相同
密码强度校验	有 (大小写字母+数字+特殊字符)	相同
首次登录强制改密	有 (<code>first_login</code> 机制)	相同
Session 版本控制	有 (改密后旧 session 失效)	相同

2.4 输入验证与 XSS 防护

比较项	v4alpha	v4beta
输入 XSS 过滤	有 (<code>sanitize_input</code>)	有
文件上传校验	有 (扩展名+MIME+魔数)	有
CSP 头部	有 (<code>default-src 'self'</code>)	有
安全响应头	有 (X-Frame-Options, XSS-Protection 等)	有

2.5 配置安全

比较项	v4alpha	v4beta
Secret Key 强度检查	有 (≥ 32 字符)	有
Session Cookie HttpOnly	是	是
Session Cookie SameSite	Lax	Lax
Debug 模式保护	是 (非交互式环境禁止启动)	是

2.6 限流保护

比较项	v4alpha	v4beta
登录频率限制	有 (IP+用户名双维度)	有
短窗口锁定	有 (60秒内3次失败锁定1分钟)	有
用户名维度锁定	有 (10次失败锁定15分钟)	有

2.7 时序攻击防护

比较项	v4alpha	v4beta
用户不存在时使用假哈希	有 (<code>DUMMY_HASH</code>)	有
评估	均受保护	均受保护

三、漏洞影响总结

影响评级

漏洞	类型	CVSS 3.1 评分	严重程度
漏洞1：水平越权查看任意用户资料	IDOR / Broken Access Control	7.5 (High)	高危
漏洞2：越权为任意用户充值/扣款	Broken Access Control	8.8 (High)	高危
漏洞3：金额校验缺失	Business Logic Flaw	6.5 (Medium)	中危

攻击链

三个漏洞可以组合利用，形成完整的攻击链：

- 信息收集**：alice 访问 `/profile?user_id=1` 获取 admin 的邮箱、手机、余额（漏洞1）
- 资金窃取**：alice 利用越权充值功能，为 admin 提交负金额请求，将 admin 余额转移给自己（漏洞2+漏洞3）
- 持续利用**：重复步骤2，直至 admin 余额耗尽或变为负数

根本原因

三个漏洞的根本原因是相同的：**服务端未对用户身份进行验证和绑定**。具体来说：

- `/profile` 和 `/recharge` 都依赖**客户端提供的用户标识**（URL 参数或表单字段），而不是**服务端会话中存储的身份信息**
- 任何身份校验的缺失都意味着：攻击者只需知道目标用户的 ID（通常是自增整数，很容易枚举），即可冒充该用户执行操作

安全设计原则

- 永远不要信任客户端输入的身份标识**：用户的身份应当从服务端管理的会话中获取
- 纵深防御**：即使有身份校验，也应当在业务逻辑层面再次确认
- 最小权限原则**：用户只能访问和操作自己的资源
- 服务端校验不可省略**：前端校验（HTML 的 min/max 属性）仅用于提升用户体验，安全校验必须在服务端完整实施

报告生成日期：2026-07-10

分析基于：v4alpha（有漏洞版）与 v4beta（安全修复版）的代码对比